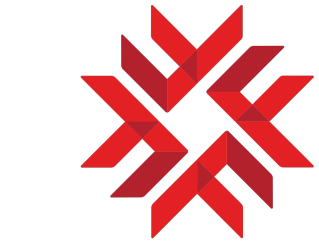
Natural Language Processing 2

Agentic AI

Fatemeh Dehrouyeh



FANSHAWE

Contents

1. Tool Calling
2. AI Agent
3. Tools
   - Claude Code

Current Section

1 Tool Calling

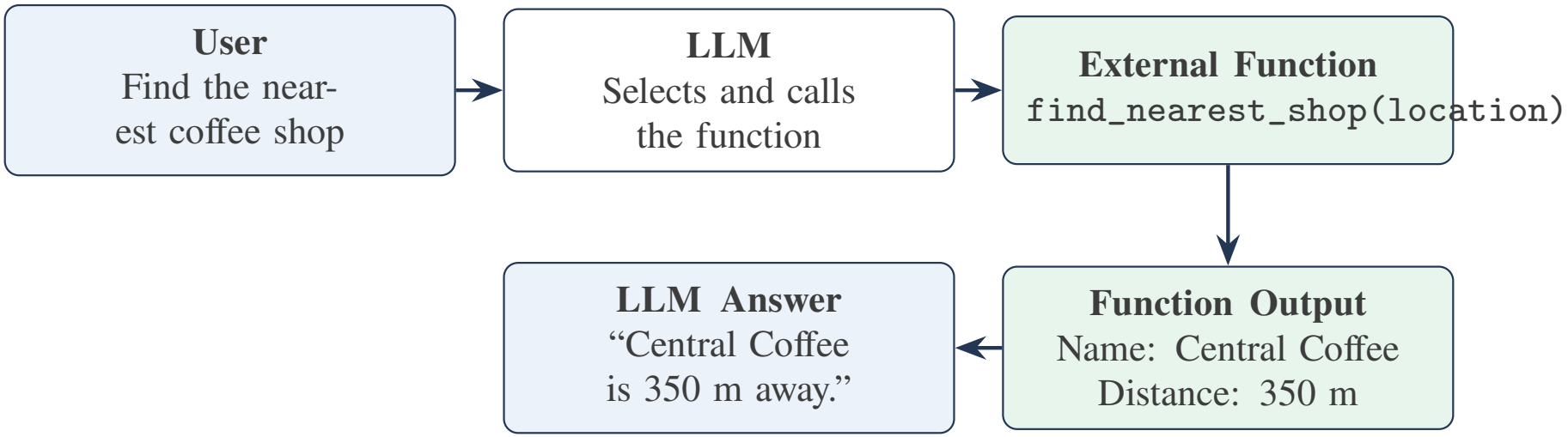
2 AI Agent

3 Tools

Introduction to Tool Calling

Definition:
Tool calling enables an LLM to use external functions with predefined inputs and outputs to retrieve information or perform tasks beyond its internal knowledge.

Example: "Find the nearest coffee shop to me."



Key idea: The LLM decides when to call a function, provides the required inputs, and uses its output to generate an answer.

How Tool Calling Works

Example: "Find the nearest coffee shop to me."

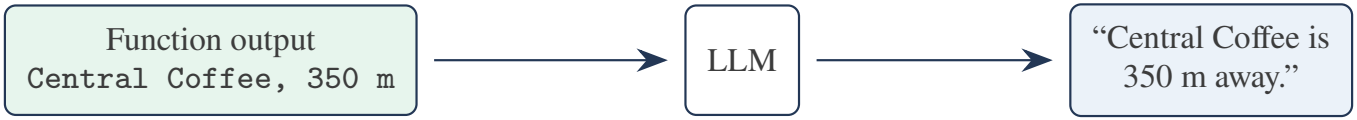
1. LLM selects a function and prepares its arguments



2. Backend executes the function



3. Function result returns to the LLM



Important: The LLM sees the function definition, arguments, and returned results. The backend implementation is hidden from the LLM.

Current Section

1 Tool Calling

2 AI Agent

3 Tools

What is an AI Agent?

Definition:
An **AI agent** is a system that autonomously pursues goals, makes decisions, and performs tasks on behalf of a user.

Example: "Find the nearest coffee shop and give me walking directions."

- **Understand the goal:** Identify what the user wants to accomplish.
- **Plan:** Determine which steps and tools are needed.
- **Act:** Use location and map tools to find nearby coffee shops and calculate a walking route.
- **Evaluate:** Check whether the results satisfy the user's request.
- **Respond:** Present the nearest coffee shop and walking directions.

Key distinction: Tool calling allows an LLM to execute individual functions. An AI agent can coordinate multiple tool calls and decisions to achieve a broader goal.

Executive message

Agentic AI shifts AI from **answering** to **acting**.

- It plans multi-step tasks.
- It calls tools and software systems.
- It checks progress against goals.
- It asks humans for approval when risk is high.

Core framing synthesized from IBM, MIT Sloan, Anthropic, OpenAI, and NIST sources.

ReAct: Reason + Act

ReAct is an approach where an AI agent alternates between reasoning and taking actions to achieve a goal.

How it works:

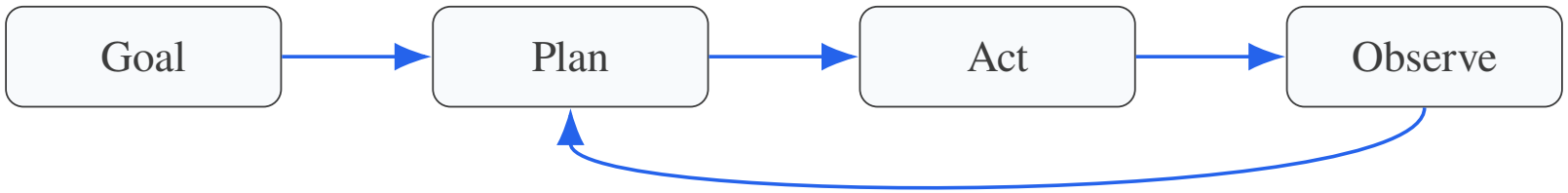
- **Observe:** Understand the user's request and available information.
- **Plan:** Decompose a complex task into smaller, manageable steps.
- **Act:** Execute the next step using available tools.
- **Repeat:** Observe the results, update the plan, and continue until the goal is achieved.

Key idea: Instead of solving a complex task in one step, the agent iteratively plans, acts, and observes until it reaches the desired outcome.

What is agentic AI?

Practical definition

An agentic AI system can pursue a goal with limited supervision by perceiving context, reasoning over next steps, using tools, and adapting its actions.



IBM defines agentic AI as systems that can accomplish goals with limited supervision. MIT Sloan describes agents as systems that perceive, reason, and act.

Why it matters now

Before

- Human writes every step.
- AI returns content.
- User copies output into tools.
- Automation breaks when context changes.

Now

- User defines the outcome.
- Agent selects actions.
- Tools execute work.
- Feedback loops correct course.

The key change

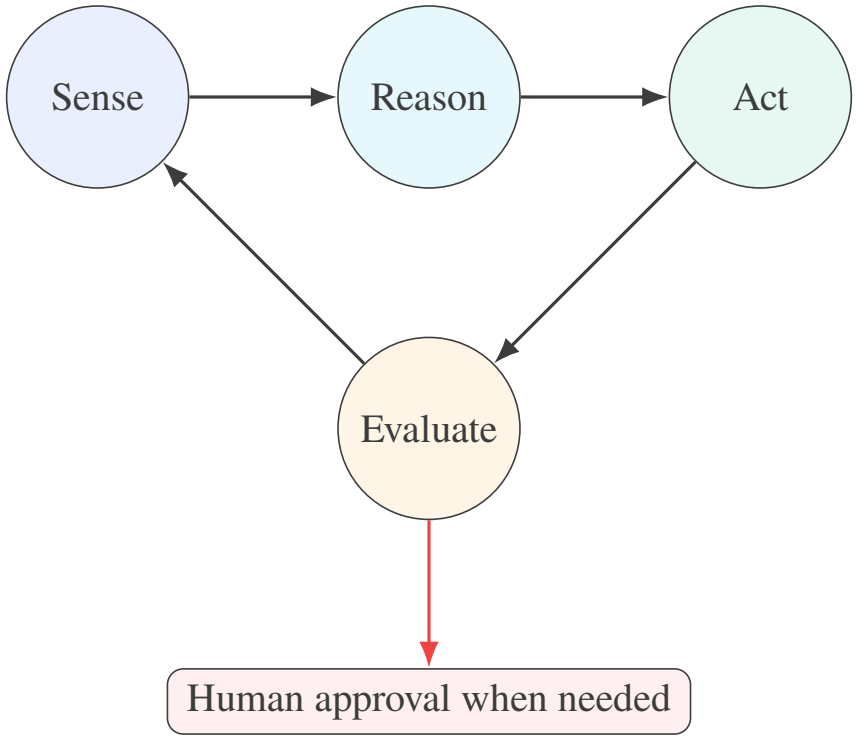
The interface becomes intent, not instructions.

Generative AI vs agentic AI

	Generative AI	**Agentic AI**
Main output	Text, code, images, summaries	Completed tasks and decisions
User role	Prompt and review	Set goal, constraints, approval rules
System role	Generate response	Plan, use tools, observe, adapt
Risk point	Wrong content	Wrong action
Best metric	Quality of answer	Quality of outcome

IBM distinguishes agentic AI by its focus on decisions and action rather than content generation alone.

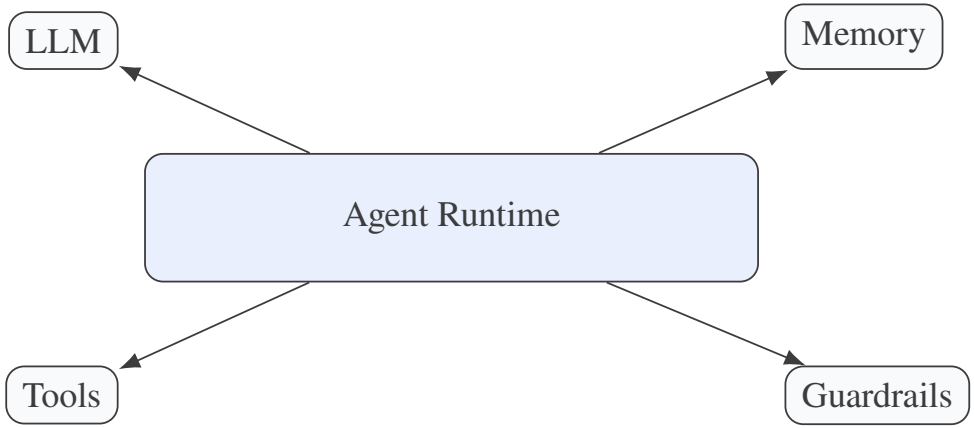
The agent loop



- Sense: read state, context, documents, APIs.
- Reason: choose next step.
- Act: call a tool or update a system.
- Evaluate: check if the goal is met.

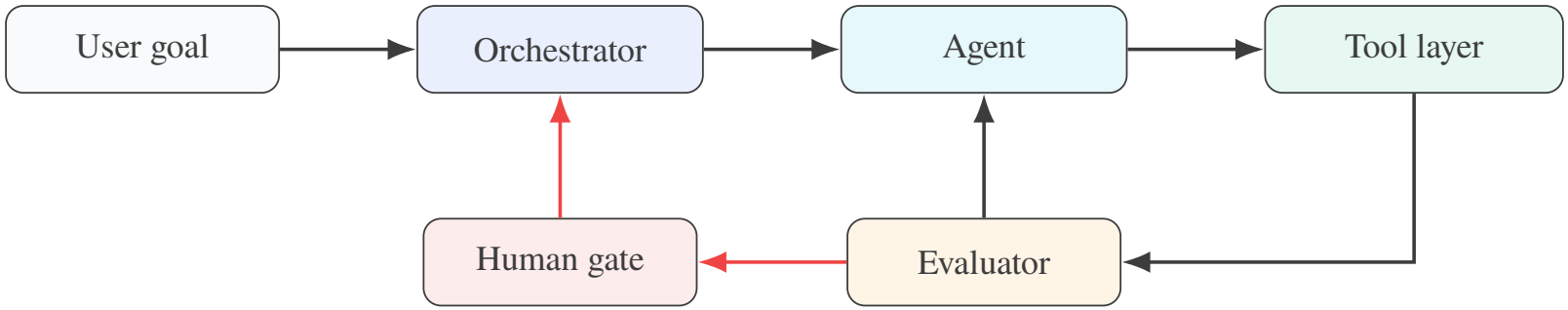
Core components

- **Model**: interprets goals and context.
- **Tools**: APIs, databases, browsers, code, email, CRM.
- **Memory**: task state, user preferences, history.
- **Planner**: decomposes work into steps.



OpenAI describes agents as applications that plan, call tools, collaborate across specialists, and keep state.

Agentic architecture



Common agent patterns

1. **Prompt chaining**: break work into verified stages.
2. **Routing**: send tasks to specialist agents.
3. **Tool use**: call APIs, code, search, or databases.
4. **Evaluator-optimizer**: generate, critique, revise.
5. **Multi-agent collaboration**: agents divide subtasks.

Anthropic recommends simple, composable patterns before complex frameworks.

Workflows vs agents

Workflow

- Predefined steps.
- Predictable path.
- Easier to test.
- Best for stable processes.

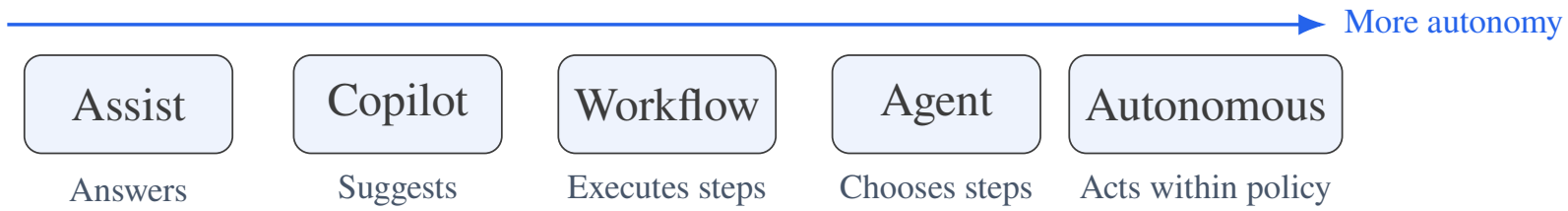
Agent

- Dynamic step selection.
- Adapts to context.
- Harder to verify.
- Best for variable tasks.

Rule

Use the least autonomy that gets the job done.

Autonomy levels



- More autonomy increases leverage.
- More autonomy also increases governance needs.

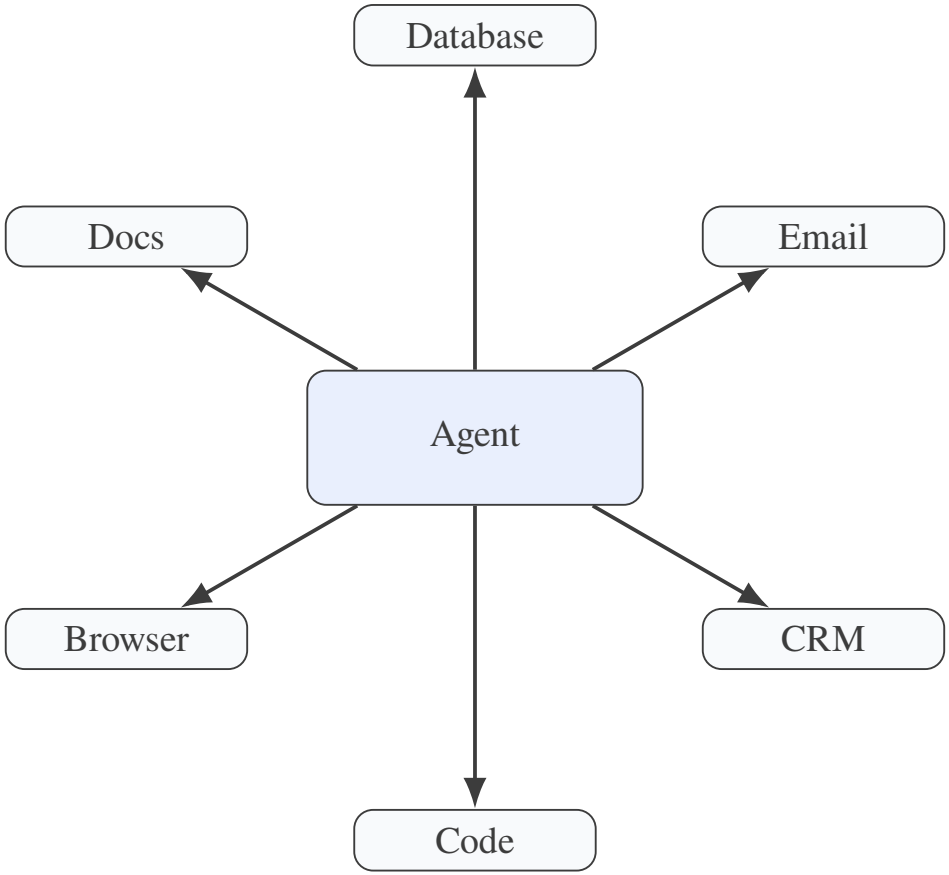
Copilot example: GitHub Copilot suggests code while the programmer decides what to accept, edit, or run.

Autonomy levels

Agentic AI systems can be classified by how much independence they have.
At the lowest level, an assistant answers questions. The human remains responsible for deciding and acting.
A copilot provides suggestions and helps complete parts of a task, but the human still controls the workflow.
A workflow system can execute predefined steps. It follows a fixed process and works well when the task is predictable.
An agent can choose the next step based on context. It can adapt when the situation changes.
At the highest level, an autonomous system can act within a defined policy. It has the most leverage, but it also requires the strongest governance.

The main lesson is simple: more autonomy can create more value, but it also increases the need for oversight, testing, permissions, and human approval.

Tool use is the power source



Design principle

Every tool needs permissions, logging, validation, and failure handling.

Current Section

1 Tool Calling

2 AI Agent

3 Tools
- Claude Code

Current Subsection

1 Tool Calling

2 AI Agent

3 Tools
- Claude Code

What is Claude Code?

Claude Code is Anthropic's coding agent. It is one of the most popular tools.

Use it when you want help with coding, debugging, explaining code, and working inside a project. It runs in the terminal and helps you work with code projects.

It can:

- Read files in your project
- Explain code
- Suggest changes
- Edit code
- Run commands

Installing Claude Code

Claude Code must be installed first before using.

1. Go to the official Claude Code quickstart page:
   `https://code.claude.com/docs/en/quickstart`
2. Choose your operating system.
3. Copy the installation command from the official page.
4. Paste it into your terminal.
5. Check the installation:
   `claude -version`

Installation

```
PS C:\Users\Aurora> irm https://claude.ai/install.ps1 | iex
Setting up Claude Code...

✔Claude Code successfully installed!

  Version: 2.1.168

  Location: C:\Users\Aurora\.local\bin\claude.exe


  Next: Run claude --help to get started

⚠Setup notes:
  ● Native installation exists but C:\Users\Aurora\.local\bin is not in your PATH
. Add it by opening: System Properties
    → Environment Variables → Edit User PATH → New → Add the path above. Then res
tart your terminal.


✅ Installation complete!
```

Starting Claude Code

After installation, open a terminal inside your project folder, or change the terminal directory to your project folder: `cd my-python-project`

Then start Claude Code:
`claude (or complete path)`

Claude Code can now see the files in that folder.

You can ask it questions such as:

- *Explain this project in simple words.*
- *What does main.py do?*
- *Find possible bugs in this code.*
- *Suggest one improvement, but do not edit files yet.*

Simple Claude Code Example

Suppose we have a file called `calculator.py`:

```
def add(a, b):
    return a + b
```

Inside Claude Code, we can ask:

Explain this file. Then add a subtract function.

Claude Code can:

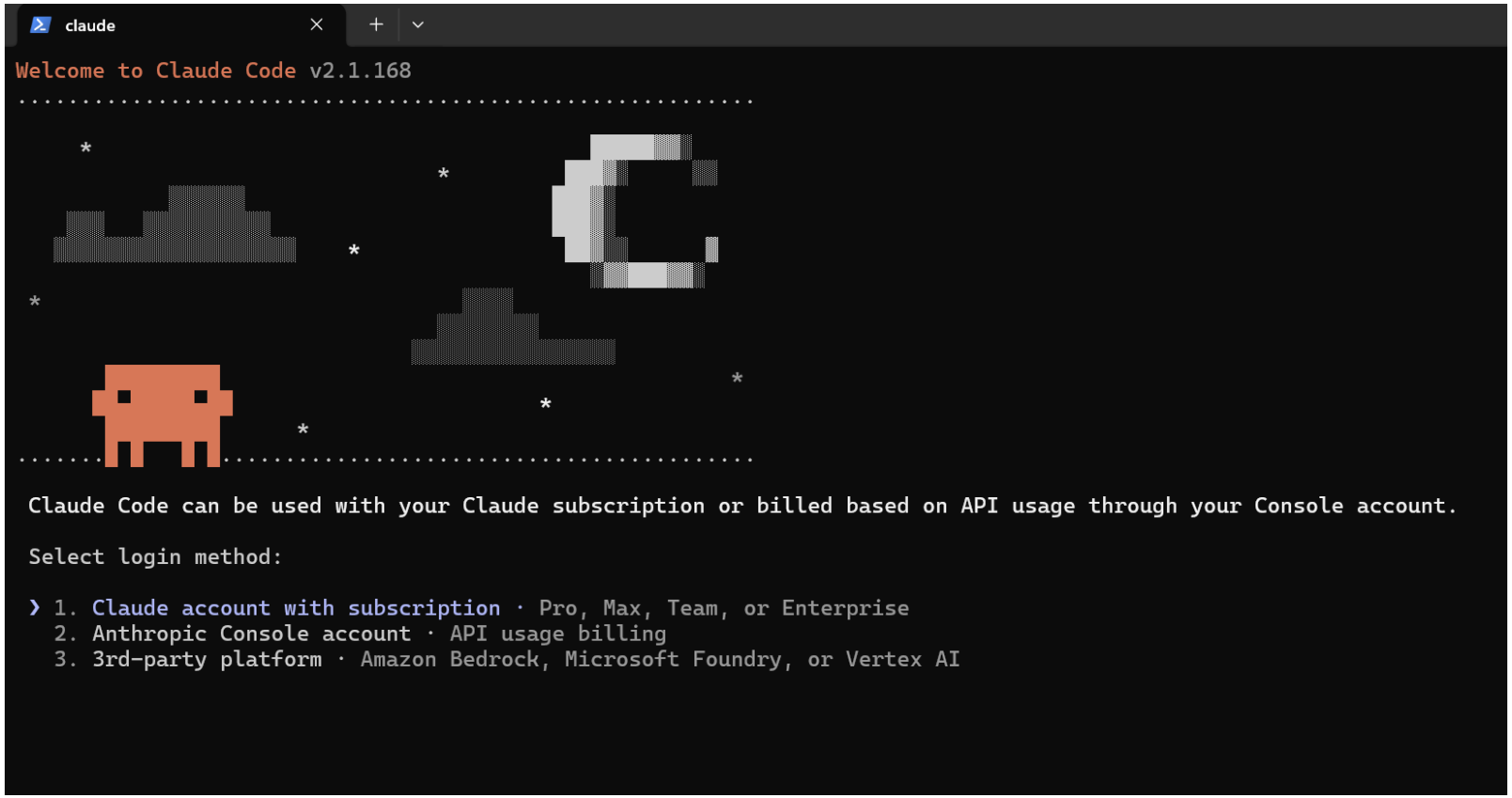
- Read the file
- Explain the function
- Suggest a code change
- Edit the file if we approve

This is different from normal chat because Claude Code works inside the project folder.

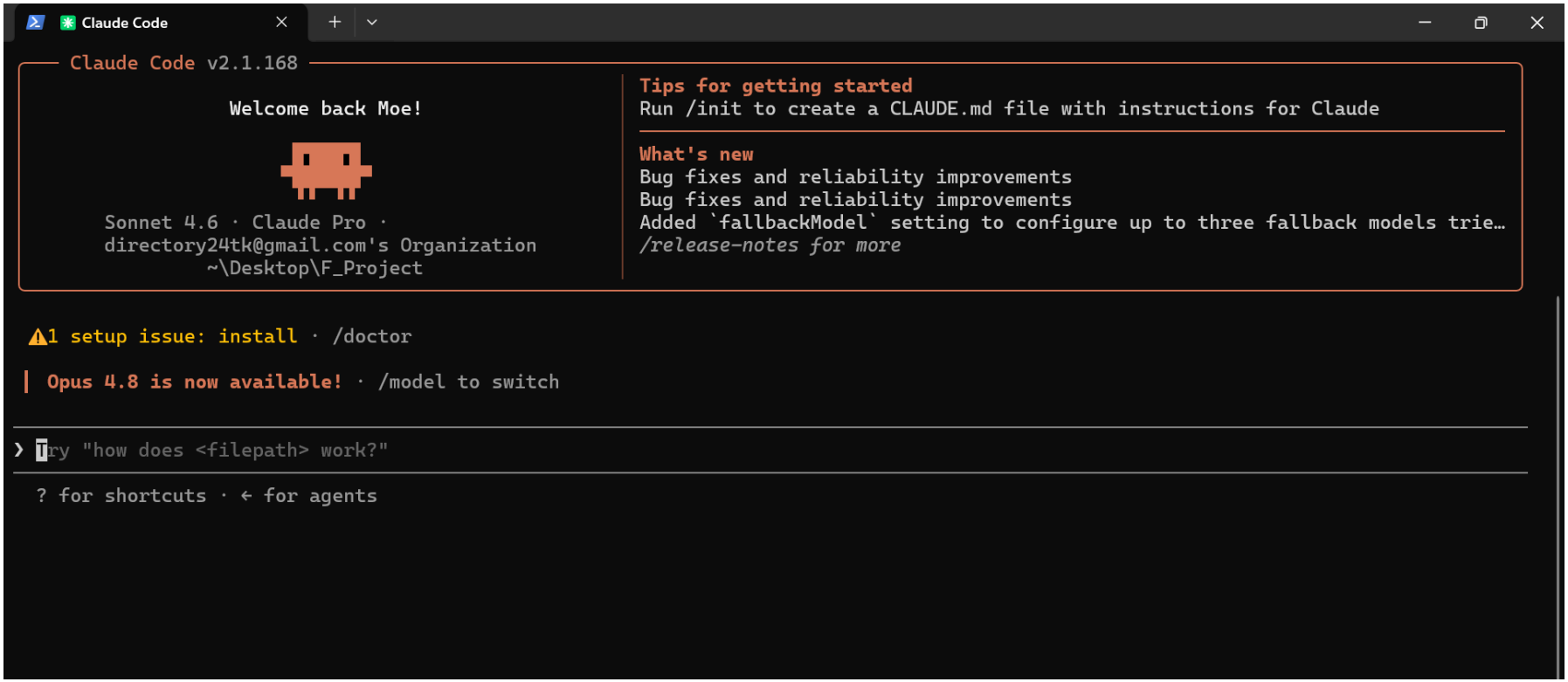
Starting Claude in Project Folder



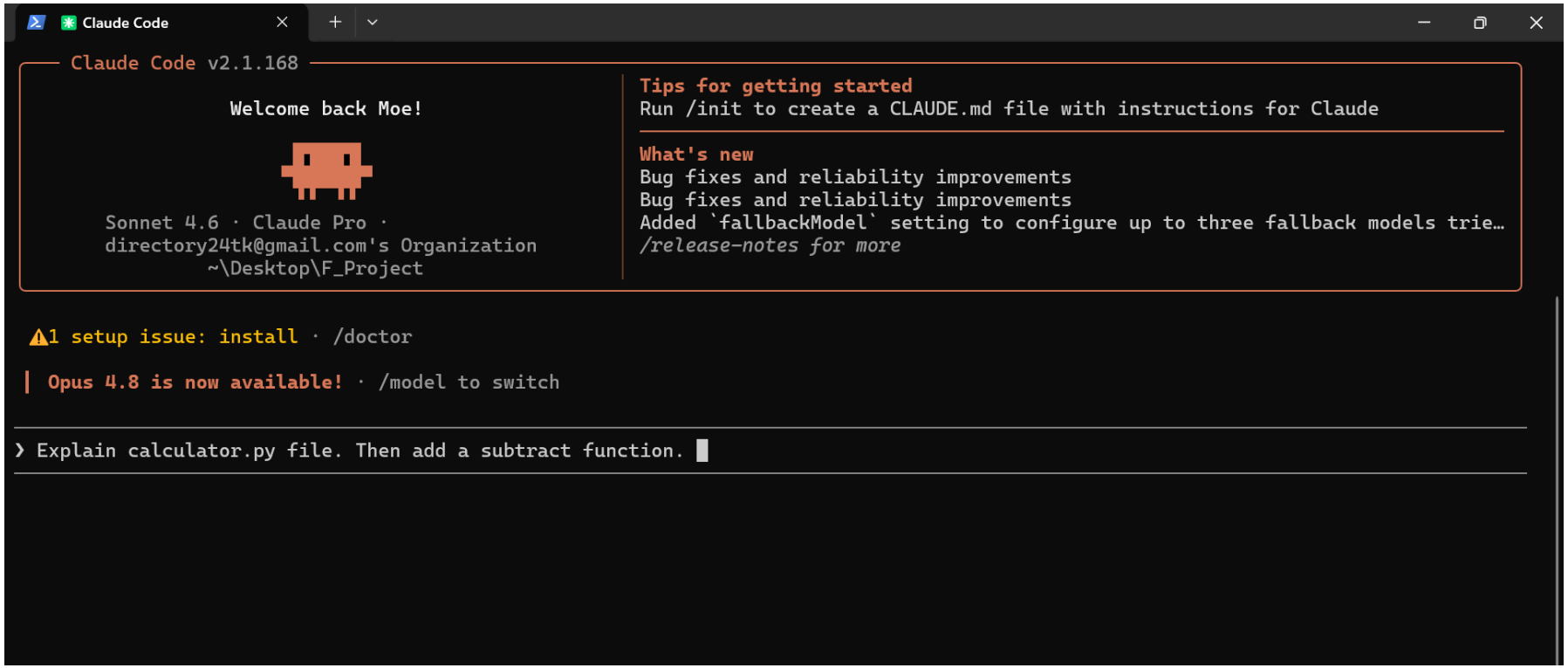
Subscription is Needed



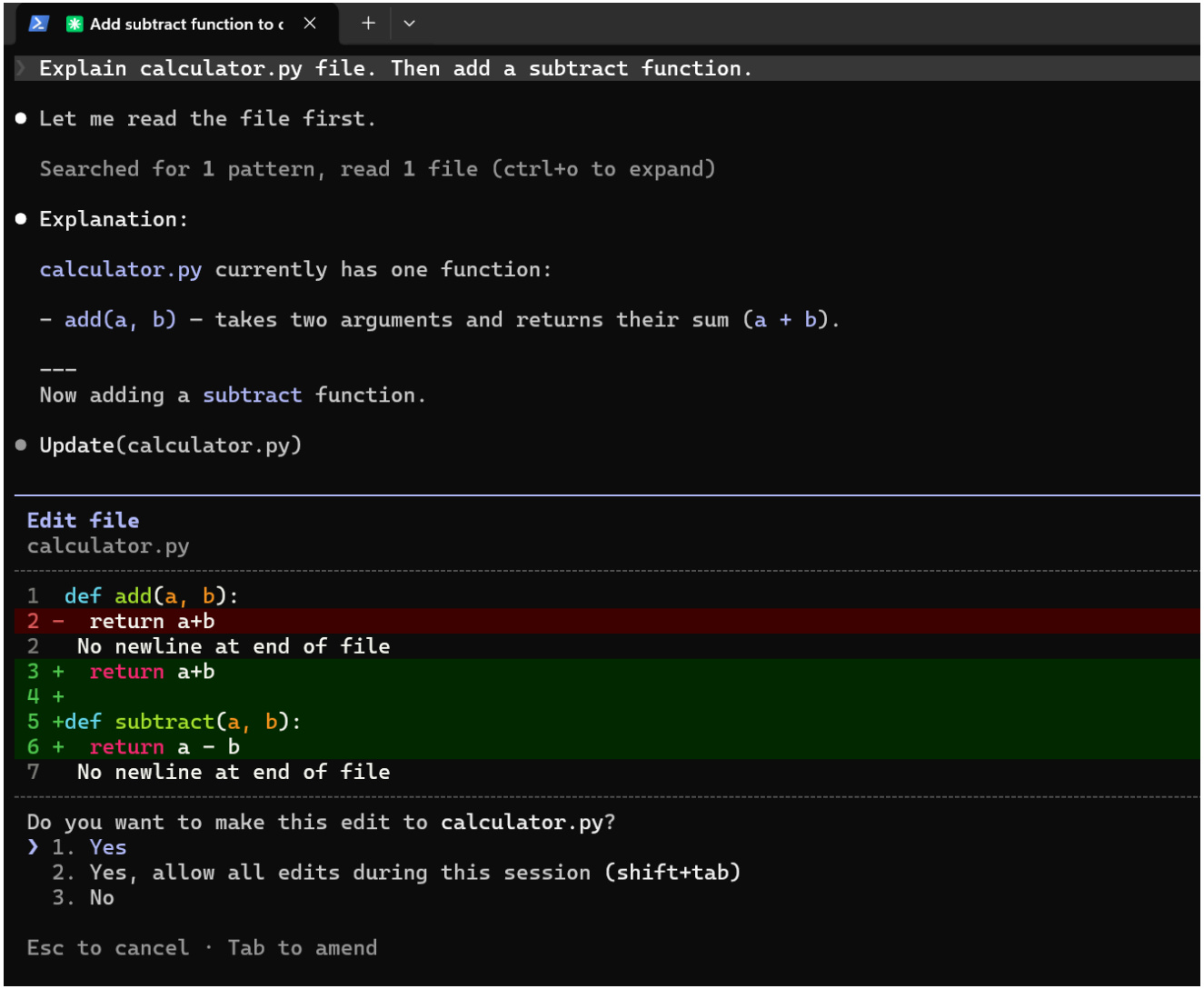
Logging In


Claude Code
Claude Code v2.1.168
Welcome back Moe!
Sonnet 4.6 · Claude Pro ·
directory24tk@gmail.com's Organization
~\Desktop\F_Project
Tips for getting started
Run /init to create a CLAUDE.md file with instructions for Claude
What's new
Bug fixes and reliability improvements
Bug fixes and reliability improvements
Added `fallbackModel` setting to configure up to three fallback models trie…
/release-notes for more
1 setup issue: install · /doctor
Opus 4.8 is now available! · /model to switch
Try "how does <filepath> work?"
? for shortcuts · ← for agents

Inputting Prompt

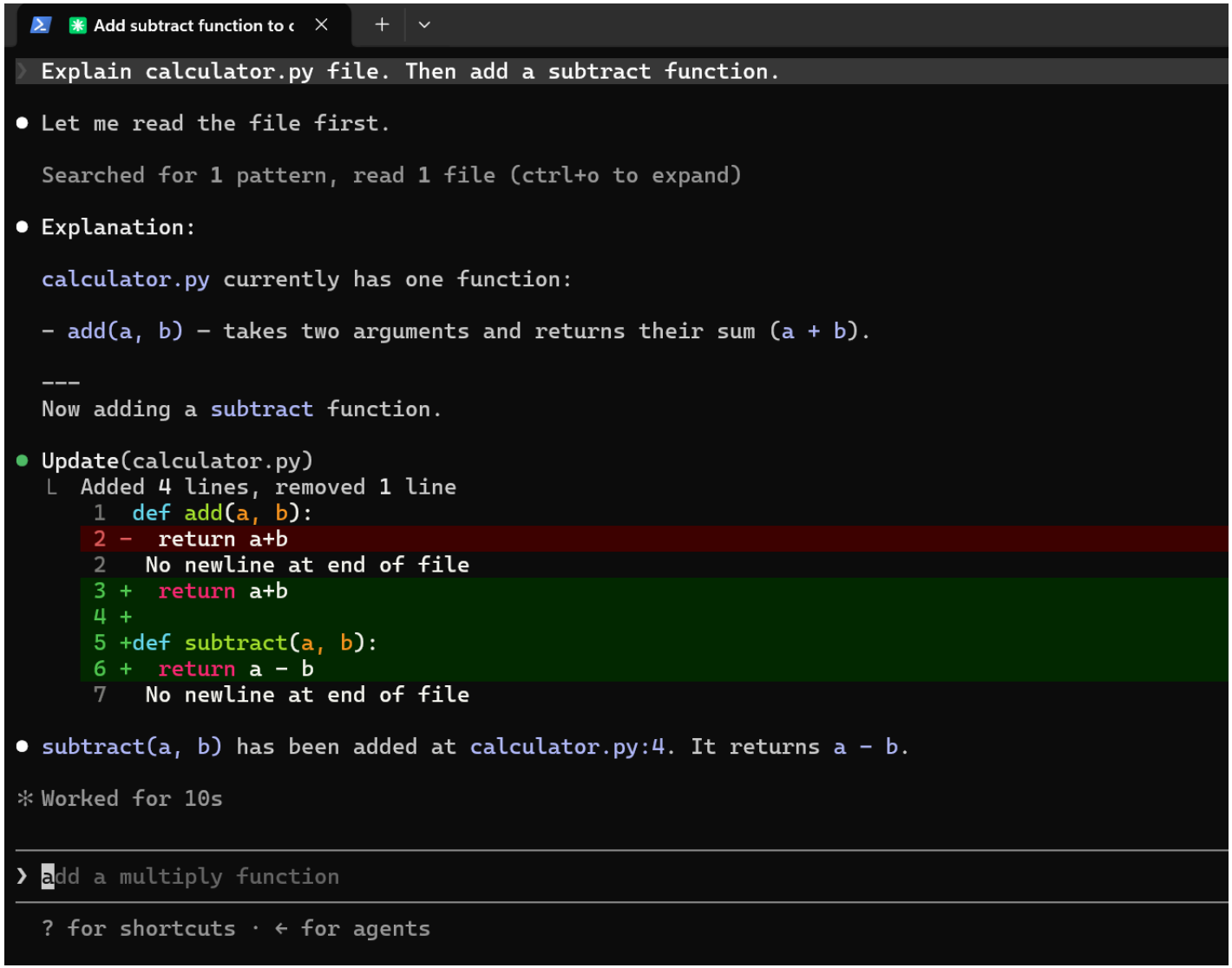


Result

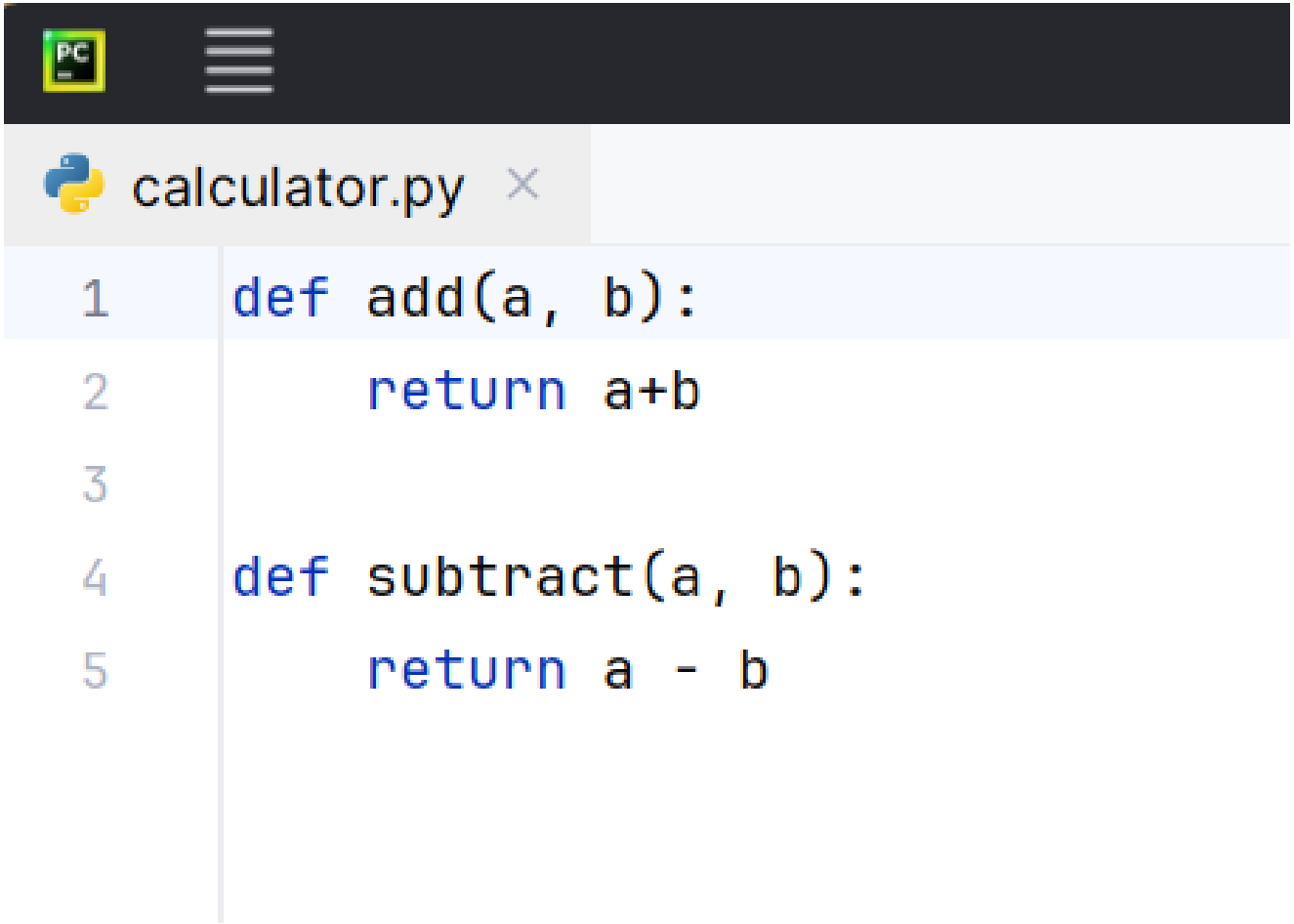


You can accept the edits.

Subtract Function Added



And you can See the Change



Examples of AI Agents

There are many agentic AI tools. They differ mainly by task, environment, and level of autonomy.

Agent	Used for
Claude Code	Terminal-based coding agent for understanding a codebase, editing files, running commands, and handling Git workflows.
OpenAI Codex	Coding agent for delegating programming tasks, changing code, running code, and working from the terminal or Codex app.
GitHub Copilot CLI	GitHub-native terminal agent for planning tasks, editing code, debugging, and working with issues or pull requests.
OpenCode	Open-source coding agent for terminal, IDE, or desktop workflows. Useful for code editing, project understanding, and multi-session coding.
Factory Droid	Software engineering agent for IDE and terminal workflows, used for refactoring, debugging, code generation, testing, and deployment.
OpenClaw	General-purpose personal agent, useful for automation, research, files, and daily productivity tasks.
Hermes Agent	Experimental open-source agent model/tooling, useful for testing autonomous agent behavior and local AI workflows.

Congratulations!

Now you know how to work with agentic AI tools and use them for real tasks!

